Programming for AI	Lab 06 **NumPy: High-Performance Numerical Computing and N-Dimensional Arrays**

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES

Fall 2026

AIMS AND OBJECTIVES

- To understand NumPy and its role in high-performance numerical computing with Python.
- To learn how to create, inspect, reshape, index, slice, and manipulate N-dimensional (ndarray) arrays.
- To understand NumPy data types, dimensions, shapes, axes, and memory-efficient numerical storage.
- To perform element-wise arithmetic, comparison, aggregation, and mathematical operations without explicit Python loops.
- To use broadcasting, vectorization, Boolean indexing, and conditional selection effectively.
- To work with random numbers, sorting, searching, stacking, splitting, copying, and views.
- To understand the difference between Python lists and NumPy arrays and select the appropriate structure for AI/data-processing tasks.
- To prepare numerical data for machine learning and AI workflows.

INTRODUCTION

NumPy (Numerical Python) is a fundamental Python library for scientific computing, data analysis, engineering, and artificial intelligence. Its core data structure is the ndarray, an N-dimensional array that stores values efficiently and supports fast numerical operations. Unlike ordinary Python lists, NumPy arrays are designed for homogeneous numerical data and allow operations to be applied to complete arrays without writing explicit loops.
In this lab, students will learn the complete foundation of NumPy array manipulation: installation/import, array creation, dimensions, shape, size, data type, indexing, slicing, reshaping, flattening, concatenation, stacking, splitting, arithmetic, comparisons, aggregations, broadcasting, vectorization, conditional operations, random-number generation, sorting, searching, copying/views, and basic linear-algebra operations. These skills form the foundation for later work with pandas, scikit-learn, deep learning frameworks, and other AI tools.

INSTALLING AND IMPORTING NUMPY

A NumPy array is a data structure used to store numerical values efficiently. It can have one or multiple dimensions and allows fast mathematical operations on large amounts of data.

If NumPy is not already installed, install it using:

```
!pip install numpy
```

```
import numpy as np
#Check the installed version:
import numpy as np
print(np.__version__)
```

PYTHON LISTS VS NUMPY ARRAYS

Feature	Python List	NumPy ndarray
Data types	Can contain mixed types	Normally homogeneous
Numerical operations	Often require loops	Vectorized operations
Performance	General-purpose	Optimized for numerical work
Dimensions	Nested lists	Native N-dimensional structure
Memory	Generally more overhead	More compact for numerical data
AI/Data Science	Useful for general collections	Core numerical foundation

```
numbers = [1, 2, 3, 4]
arr = np.array([1, 2, 3, 4])
```

```
print(numbers * 2)   # list repetition
print(arr * 2)       # element-wise multiplication
```

3. CREATING NUMPY ARRAYS

3.1 From a Python List

You can create arrays from Python lists or generate them using NumPy functions.
Arrays can have 1, 2, 3, or more dimensions.

```
arr = np.array([10, 20, 30, 40])
print(arr)
```

3.2 One-, Two-, and Three-Dimensional Arrays

```
a1 = np.array([1, 2, 3])

a2 = np.array([[1, 2, 3],
               [4, 5, 6]])

a3 = np.array([[[1, 2], [3, 4]],
               [[5, 6], [7, 8]]])

print(a1)
print(a2)
print(a3)
```

A 1-D array has one axis, a 2-D array has rows and columns, and a 3-D array contains multiple 2-D arrays.

3.3 Arrays of Zeros and Ones

`np.zeros()` creates an array where every value is `0`.
`np.ones()` creates an array where every value is `1`.

These are useful when you need an array with a predefined shape.

```
zeros = np.zeros(5)
ones = np.ones((2, 3))
print(zeros)
print(ones)
```

3.4 Full, Empty, and Identity Arrays

`np.full()` creates an array filled with a value you specify.
`np.empty()` creates an array without initializing its values, so don't rely on its initial contents.
`np.eye()` creates an identity matrix with `1`s on the main diagonal.

```
a = np.full((2, 3), 7)
```

```
b = np.empty((2, 3))
I = np.eye(3)

print(a)
print(b)
print(I)
```

np.empty creates an array without initializing its values; its contents should not be assumed until assigned.

3.5 arange()

`np.arange()` generates numbers within a specified range.
You can also specify the step between numbers.
For example, `np.arange(0, 10, 2)` produces `0, 2, 4, 6, 8`.

```
a = np.arange(0, 10, 2)
print(a)  # [0 2 4 6 8]
```

3.6 linspace()

`np.linspace()` generates a specific number of evenly spaced values.
For example, `np.linspace(0, 1, 5)` gives five equally spaced values between 0 and 1.
It is particularly useful in numerical calculations and plotting.

```
a = np.linspace(0, 1, 5)
print(a)
arr = np.array([[10, 20, 30], [40, 50, 60]], dtype=np.int32)

print(arr.ndim)
print(arr.shape)
print(arr.size)
print(arr.dtype)
print(arr.itemsize)
print(arr.nbytes)
```

4. ARRAY ATTRIBUTES

Array attributes give you information **about an array** without changing it.
For example, you can find its dimensions, shape, data type, and memory usage.
Important attributes include `ndim`, `shape`, `size`, `dtype`, `itemsize`, and `nbytes`.

Attribute	Meaning
ndim	Number of dimensions / axes
shape	Size along each dimension
size	Total number of elements
dtype	Data type of array elements
itemsize	Bytes used by one element
nbytes	Total bytes used by the array data

5. NUMPY DATA TYPES (dtype)

NumPy provides explicit numerical data types. Common types include int32, int64, float32, float64, bool, and complex types. Choosing the appropriate type can affect memory usage and performance.

```
a = np.array([1, 2, 3], dtype=np.int32)
b = np.array([1.5, 2.5, 3.5], dtype=np.float64)
c = np.array([True, False, True], dtype=bool)

print(a.dtype)
print(b.dtype)
print(c.dtype)
```

5.1 Type Conversion

You can convert an array from one data type to another using `.astype()`.
For example, floating-point values can be converted into integers.
When converting floats to integers, the fractional part is discarded.

```
arr = np.array([1.2, 2.8, 3.9])
integers = arr.astype(np.int32)
print(integers)
```

Be careful: converting floating-point values to integer types discards the fractional component.

6. INDEXING

Indexing means accessing a specific element of an array.
Like Python lists, NumPy uses zero-based indexing.
For multidimensional arrays, you provide an index for each dimension.

6.1 One-Dimensional Indexing

You access an element using its position, such as `arr[0]`.
`arr[-1]` accesses the last element.
This works similarly to indexing a normal Python list.

```
arr = np.array([10, 20, 30, 40, 50])
print(arr[0])
print(arr[-1])
```

6.2 Two-Dimensional Indexing

For a 2-D array, you use `arr[row, column]`.
The first index selects the row and the second selects the column.
For example, `arr[1, 2]` means row 1, column 2.

```
arr = np.array([[10, 20, 30],
```

```
              [40, 50, 60]])

print(arr[0, 1])
print(arr[1, 2])
```

The first index selects the row and the second selects the column.

6.3 Three-Dimensional Indexing

A 3-D array requires three indexes.
They identify the position along the first, second, and third dimensions.
For example, `arr[1, 2, 3]` selects one specific element.

```
arr = np.arange(24).reshape(2, 3, 4)
print(arr[1, 2, 3])
```

7. SLICING

Slicing allows you to select a **range of elements** instead of one element.
For example, `arr[1:5]` selects elements from index 1 up to, but not including, 5.
You can also slice rows and columns in multidimensional arrays.

Reshaping changes the **structure/shape** of an array without changing its data.
For example, 12 elements can be changed from a 1-D array into a `3 × 4` matrix.
The total number of elements must remain the same.

```
arr = np.array([10, 20, 30, 40, 50, 60])
print(arr[1:5])
print(arr[:3])
print(arr[3:])
print(arr[::2])

#Two-dimensional slicing:

arr = np.arange(1, 13).reshape(3, 4)
print(arr[0:2, 1:3])
print(arr[:, 0])
print(arr[1, :])
```

8. CHANGING ARRAY VALUES

```
arr = np.array([10, 20, 30, 40])
arr[1] = 99
print(arr)

matrix = np.array([[1, 2], [3, 4]])
matrix[0, 1] = 20
print(matrix)
```

8.1 Assigning to a Slice

```
arr = np.array([1, 2, 3, 4, 5])
arr[1:4] = 0
```

```
print(arr)
```

9. RESHAPING ARRAYS

```
arr = np.arange(1, 13)
matrix = arr.reshape(3, 4)
print(matrix)
print(matrix.shape)
```

The number of elements must remain the same. For example, 12 elements can be reshaped to 3×4, 2×6, 4×3, or 12×1.

9.1 reshape(-1)

```
arr = np.arange(12)
matrix = arr.reshape(3, -1)
print(matrix)
```

A dimension of -1 lets NumPy infer that dimension automatically.

9.2 Flatten and Ravel

Both `flatten()` and `ravel()` convert a multidimensional array into 1-D.
`flatten()` normally creates a separate copy of the data.
`ravel()` generally returns a view when possible, so changes can affect the original array.

```
arr = np.array([[1, 2], [3, 4]])
print(arr.flatten())
print(arr.ravel())
```

flatten() returns a copy. ravel() generally returns a view when possible, so changes may affect the original array.

10. TRANSPOSE

Transpose changes rows into columns and columns into rows.
For example, a `2 × 3` matrix becomes a `3 × 2` matrix.
You can use `arr.T` or `np.transpose(arr)`.

```
arr = np.array([[1, 2, 3],
                [4, 5, 6]])
print(arr.T)
print(np.transpose(arr))
```

11. CONCATENATION AND STACKING

11.1 concatenate()

`np.concatenate()` joins arrays together along an existing axis.
For 1-D arrays, it simply places one array after the other.
For multidimensional arrays, you can specify the axis.

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(np.concatenate((a, b)))
```

11.2 vstack() and hstack()

`np.concatenate()` joins arrays together along an existing axis.
For 1-D arrays, it simply places one array after the other.
For multidimensional arrays, you can specify the axis.

```
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])

print(np.vstack((a, b)))
print(np.hstack((a, b)))
```

11.3 stack()

`np.concatenate()` joins arrays together along an existing axis.
For 1-D arrays, it simply places one array after the other.
For multidimensional arrays, you can specify the axis.

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(np.stack((a, b), axis=0))
print(np.stack((a, b), axis=1))
```

12. SPLITTING ARRAYS

Splitting is the opposite of combining arrays.
`np.split()` divides an array into multiple smaller arrays.
`hsplit()` splits horizontally, while `vsplit()` splits vertically.

```
arr = np.arange(12)
print(np.split(arr, 3))

matrix = np.arange(12).reshape(3, 4)
print(np.hsplit(matrix, 2))
print(np.vsplit(matrix, 3))
```

13. ELEMENT-WISE ARITHMETIC

NumPy performs arithmetic on corresponding elements of arrays.
For example, `a + b` adds the first element of `a` to the first element of `b`.
The same applies to subtraction, multiplication, division, powers, etc.

```
a = np.array([1, 2, 3, 4])
b = np.array([10, 20, 30, 40])
```

```
print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a ** 2)
print(b % 3)
```

These operations are element-wise. NumPy applies the operation to corresponding elements.

14. UNIVERSAL FUNCTIONS (ufuncs)

Ufuncs are NumPy's optimized mathematical functions that operate element-by-element.
Examples include `np.sqrt()`, `np.square()`, `np.exp()`, `np.log()`, and `np.sin()`.
They allow mathematical operations on entire arrays without writing loops.

NumPy provides optimized mathematical functions that operate element-wise on arrays.

```
arr = np.array([1, 4, 9, 16])

print(np.sqrt(arr))
print(np.square(arr))
print(np.exp(arr))
print(np.log(arr))
print(np.sin(arr))
```

15. AGGREGATION FUNCTIONS

Aggregation functions reduce many values into a summary value.
Examples include sum, mean, median, minimum, maximum, standard deviation, and variance.
They are commonly used for analyzing datasets and numerical results

Function	**Purpose**
np.sum()	Sum of elements
np.mean()	Arithmetic mean
np.median()	Median
np.min()	Minimum
np.max()	Maximum
np.std()	Standard deviation
np.var()	Variance
np.argmin()	Index of minimum
np.argmax()	Index of maximum
np.prod()	Product of elements

```
scores = np.array([70, 80, 90, 60, 100])

print(np.sum(scores))
```

```
print(np.mean(scores))
print(np.median(scores))
print(np.min(scores))
print(np.max(scores))
print(np.std(scores))
print(np.argmax(scores))
```

16. AXIS IN NUMPY

The axis parameter controls the direction along which an operation is performed. For a 2-D array, axis=0 operates down the rows (column-wise result), while axis=1 operates across columns (row-wise result).

```
marks = np.array([[70, 80, 90],
                  [60, 75, 85],
                  [90, 95, 88]])

print(np.mean(marks, axis=0))
print(np.mean(marks, axis=1))
print(np.sum(marks, axis=0))
print(np.sum(marks, axis=1))
```

17. COMPARISON AND BOOLEAN ARRAYS

You can compare every element of an array with a value using operators like >, <, ==, and `!=`.
NumPy returns a Boolean array containing `True` or `False` for each element.
This is the foundation of filtering data.

```
arr = np.array([10, 20, 30, 40, 50])
print(arr > 25)
print(arr == 30)
print(arr != 20)
```

18. BOOLEAN INDEXING / FILTERING

Boolean indexing allows you to select only elements that satisfy a condition.
For example, `arr[arr > 25]` returns only values greater than 25.
This is extremely useful for filtering datasets.

```
arr = np.array([10, 15, 22, 31, 40, 55])
print(arr[arr > 25])

mask = arr % 2 == 0
print(arr[mask])
```

18.1 Multiple Conditions

```
arr = np.array([10, 20, 30, 40, 50, 60])
result = arr[(arr >= 20) & (arr <= 50)]
print(result)
```

Use NumPy's &, |, and ~ for element-wise Boolean conditions. Put each condition in parentheses.

19. np.where()

```
marks = np.array([45, 70, 32, 88, 55])
result = np.where(marks >= 50, "Pass", "Fail")
print(result)
```

np.where(condition, value_if_true, value_if_false) performs vectorized conditional selection.

20. BROADCASTING

Broadcasting allows NumPy to perform operations between arrays with compatible shapes.
For example, you can add a single number to every element of a matrix.
You can also add a 1-D row to every row of a 2-D array.

Simple idea: NumPy automatically makes the shapes compatible conceptually without unnecessarily copying the smaller array.

```
arr = np.array([[1, 2, 3],
                [4, 5, 6]])
print(arr + 10)

row = np.array([10, 20, 30])
print(arr + row)
```

A common broadcasting case is adding a 1-D array to each row of a 2-D array. Broadcasting does not mean that NumPy physically creates unnecessary copies of the smaller array.

21. RANDOM NUMBERS

NumPy provides the random module for reproducible and non-reproducible random sampling.

```
rng = np.random.default_rng(42)
print(rng.random(5))
print(rng.integers(1, 11, size=5))
print(rng.normal(loc=0, scale=1, size=5))
```

Using a fixed seed such as 42 makes generated results reproducible for experiments.

21.1 Random Matrix

```
rng = np.random.default_rng(42)
```

```
matrix = rng.integers(0, 100, size=(3, 4))
print(matrix)
```

22. SORTING AND SEARCHING

`np.sort()` sorts the actual values of an array.
`np.argsort()` returns the indexes that would produce the sorted order.
`np.argmax()` and `np.argwhere()` help locate specific values or conditions.

```
arr = np.array([50, 10, 40, 20, 30])
print(np.sort(arr))
print(np.argsort(arr))
print(np.argmax(arr))
print(np.argwhere(arr > 25))
```

np.sort returns sorted values. np.argsort returns the indices that would sort the array.

23. UNIQUE VALUES

```
arr = np.array([1, 2, 2, 3, 3, 3, 4])
print(np.unique(arr))
values, counts = np.unique(arr, return_counts=True)
print(values)
print(counts)
```

24. COPYING VS VIEWS

A **view** shares the same underlying data with the original array.
Therefore, changing the view can also change the original array.
A **copy** creates independent data, so modifying it does not affect the original.

NumPy slicing can produce a view that shares data with the original array. A copy creates independent data. This distinction is important when modifying arrays.

```
original = np.array([10, 20, 30, 40])
view = original[1:3]
view[0] = 99
print(original)

original = np.array([10, 20, 30, 40])
copy = original[1:3].copy()
copy[0] = 99
print(original)
print(copy)
```

25. INSERTING, DELETING, AND APPENDING

```
arr = np.array([1, 2, 3])
print(np.append(arr, 4))
print(np.insert(arr, 1, 99))
print(np.delete(arr, 1))
```

26. HANDLING MISSING-LIKE NUMERIC VALUES WITH NaN

NaN (Not a Number) is commonly used to represent missing numerical values in floating-point arrays.
NaN-aware functions can calculate statistics while ignoring NaN values.

```
arr = np.array([10.0, 20.0, np.nan, 40.0])
print(np.isnan(arr))
print(np.nanmean(arr))
print(np.nansum(arr))
```

27. LINEAR ALGEBRA BASICS

NumPy supports basic matrix and vector operations useful in AI and machine learning.

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

print(A @ B) # matrix multiplication
print(np.dot(A, B)) # dot product / matrix product for these arrays
print(np.linalg.det(A)) # determinant
print(np.linalg.inv(A)) # inverse, if A is invertible
```

The @ operator is the standard Python operator for matrix multiplication.

28. VECTOR DOT PRODUCT

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(np.dot(a, b))
print(a @ b)
```

29. VECTORIZATION AND PERFORMANCE

Vectorization means expressing numerical operations using NumPy array operations rather than interpreted Python loops. This can make numerical workloads faster and more concise because NumPy delegates many operations to optimized compiled implementations.

```
arr = np.arange(1_000_000)
result = arr * 2 + 10
print(result[:10])
```

For numerical workloads, prefer vectorized operations when practical. Performance depends on the operation, array size, memory access patterns, and the surrounding program.

LAB Tasks

1. A university stores marks for 10 students across 5 subjects in a NumPy array. Create the dataset, calculate each student's total and average, calculate each subject's average, identify the student with the highest overall average, count students whose average is at least 50, and assign Pass/Fail using np.where(). Use axis correctly and do not use explicit loops for the numerical calculations.

2. Create a 10×4 numerical feature matrix representing four input features for an AI model. Calculate the mean and standard deviation of every feature and standardize each feature using vectorized operations. Verify that the resulting feature means are approximately zero and standard deviations are approximately one. Explain how broadcasting is used.

3.Represent a grayscale image as a 2-D NumPy array with values from 0 to 255. Generate or define an image matrix, find its minimum and maximum intensity, calculate its mean intensity, create a binary mask for pixels above a threshold, and use np.where() to produce a thresholded image.

4. Create a 2-D array containing simulated sensor readings for multiple sensors over several time points. Use axis operations to calculate per-sensor mean, minimum, maximum, and standard deviation. Identify the sensor with the highest average reading and detect readings above a selected threshold.

5.Create two compatible matrices A and B. Demonstrate addition, subtraction, element-wise multiplication, matrix multiplication using @, transpose, determinant, and inverse where valid. Print shapes before each operation and explain which operations require compatible dimensions.

6.Use np.random.default_rng() with a fixed seed to simulate 1,000 experimental measurements from a normal distribution. Calculate mean, median, standard deviation, minimum, maximum, and the percentage of values inside one standard deviation of the mean. Repeat with a different seed and compare the results.

7.Create a numerical array containing valid values and NaN entries. Detect NaNs, calculate NaN-aware statistics, replace NaNs with the mean of the non-missing values, and verify that no NaN values remain.